

MC + CIF — module-level port list

All modules also have `clk` + `rst_n` (omitted). Widths parametric: `AW=addr`, `DW=AXI data`, `IDW=axid`, `512=SRAM line`, `N_RANKS/N_BANKS`, `DQ=sub-channel`. Handshake pairs `valid/ready`, `FIFO push/pop` + `full/empty` implied where a port is a FIFO iface.

CIF — AXI front end (single port)

AXI_SLAVE

in: `araddr[AW]`, `arlen[8]`, `arsize[3]`, `arid[IDW]`, `arvalid`; `awaddr`, `awlen`, `awsize`, `awid`, `awvalid`; `wdata[DW]`, `wstrb[DW/8]`, `wlast`, `wvalid`; `rready`, `bready`

out: `arready`, `awready`, `wready`; `rdata[DW]`, `rresp[2]`, `rlast`, `rid`, `rvalid`; `bresp[2]`, `bid`, `bvalid`

AR_CH

in: `araddr`, `arlen`, `arsize`, `arid`, `arvalid`

out: `arready`, `ar_req{addr,len,id}`, `ar_push`

AW_CH

in: `awaddr`, `awlen`, `awsize`, `awid`, `awvalid`

out: `awready`, `aw_req{addr,len,id}`, `aw_push`

W_CH

in: `wdata[DW]`, `wstrb[DW/8]`, `wlast`, `wvalid`

out: `wready`, `w_beat{data,strb,last}`, `wd_wr_en`

R_CH

in: `r_beat{rdata,rresp,rlast,rid}`, `r_valid`, `rready`

out: `rdata`, `rresp`, `rlast`, `rid`, `rvalid`, `r_ready`

BRESP_CH

in: `b_resp{bresp,bid}`, `b_valid`, `bready`

out: `bresp`, `bid`, `bvalid`, `b_ready`

CIF — request FIFOs / pairing

AR_REQ_FIFO

in: `ar_req`, `push`

out: `ar_req_o`, `empty`, `full`, `pop`

AW_REQ_FIFO

in: `aw_req`, `push`

out: `aw_req_o`, `empty`, `full`, `pop`

W_BURST_FIFO

in: `desc{start_ptr,count}`, `push`

out: `desc_o`, `empty`, `full`, `pop`

WR_PAIR

in: `aw_req_o`, `w_desc`

out: `wr_req{addr,wd_slot,count}`, `wr_valid`

CIF — validate / segment / hash

REQ_VAL_R

in: `ar_req`, `geom_cfg`

out: `val_req`, `n_segs`, `err`, `err_type`

REQ_VAL_W

in: `wr_req`, `geom_cfg`

out: `val_req`, `n_segs`, `err`, `err_type`

ERR_HDLR_R

in: `err`, `err_type`, `arid`

out: `rob_alloc_err`, `status`

ERR_HDLR_W

in: `err`, `err_type`, `awid`

out: `rob_alloc_err`, `status`

BURST_SPLIT_R

in: `val_req`, `n_segs`, `row_size`

out: `seg_req{addr,beats}`, `seg_valid`

BURST_SPLIT_W

in: `val_req`, `n_segs`, `wd_slot[]`

out: `seg_req{addr,beats,wd_slot}`, `seg_valid`

R_ADR_HASH

in: `seg_addr`, `hash_cfg`, `adr_cfg`

out: `daddr{rank,bg,bank,row,col}`

W_ADR_HASH

in: `seg_addr`, `hash_cfg`, `adr_cfg`

out: `daddr{rank,bg,bank,row,col}`

PKT_FORM_R

in: `daddr`, `rob_index`, `dbuf_addr`, `beats`

out: `rd_pkt{rob_index,dbuf_addr,daddr,beats}`

PKT_FORM_W

in: `daddr`, `rob_index`, `wd_slot`, `beats`

out: `wr_pkt{rob_index,wd_slot,daddr,beats}`

CIF — reorder buffers

ROB_RD

in: `alloc_req`, `resp{rob_index,pkt_num,status}`

out: `rob_index`, `start_ptr`, `exp_pkts`, `bitmap`, `rob_full`, `retire{rob_index}`

ROB_WR

in: `alloc_req`, `resp{rob_index,status}`

out: `rob_index`, `retire{rob_index,status}`, `rob_full`

CIF — data SRAM + managers (shared with MCC)

RD_SRAM

in: `wr_data[512]`, `wr_addr(dbuf)`, `wr_en` (MC port); `rd_addr`, `rd_en` (CIF port)

out: `rd_data[512]`

WD_SRAM

in: `wr_data(beats)`, `wr_addr(wd_slot)`, `wr_en` (CIF port); `rd_addr`, `rd_en` (MC port)

out: `rd_data[512]`

RD_SRAM_MGR

in: `alloc_req(size)`, `free_req`

out: `dbuf_addr`, `range`, `full`

WR_SRAM_MGR

in: `alloc_req(burst)`, `free_req(pkt)`

out: `wd_slot[]`, `full`

CIF — response path

MERGE_LOGIC

in: `rob_retire`, `bitmap`, `rd_data[512]`

out: `r_beat{rdata,rresp,rlast}`, `r_valid`, `free_range`

BRESP_GEN

in: `rob_wr_retire{rob_index,status}`

out: `b_resp{bresp,bid}`, `b_valid`

CIF — CDC to MCC

CH_R_PKT_FIFO

in: rd_pkt, push

out: rd_pkt_o, empty, full

CH_W_PKT_FIFO

in: wr_pkt, push

out: wr_pkt_o, empty, full

CH_R_REQ_AFIFO

in: rd_pkt (cif_clk), wr_en

out: rd_pkt (mc_clk), rd_valid, rd_ready, full

CH_W_REQ_AFIFO

in: wr_pkt (cif_clk), wr_en

out: wr_pkt (mc_clk), rd_valid, rd_ready, full

CH_R_RESP_AFIFO

in: resp (mc_clk), wr_en

out: resp (cif_clk), rd_valid

CH_W_RESP_AFIFO

in: resp (mc_clk), wr_en

out: resp (cif_clk), rd_valid

CIF — config

APB_SLAVE

in: paddr, pwidth, psel, penable, pwrite

out: prdata, pready, csr_wr, csr_addr, csr_wdata

CSR

in: csr_wr, csr_addr, csr_wdata

out: csr_rdata, cfg_bus

CFG_REG / ADR_CFG / GEOM_CFG / HASH_CFG

in: cfg_bus (write)

out: enable/mode; field pos/width; ranks/bg/banks/rows/cols/row_size; hash taps (bc)

MCC — ingress (per rank)

R_REQ_AFIFO / W_REQ_AFIFO

in: pkt (cif_clk), wr_en

out: pkt (mc_clk), valid, ready, full

RD_BP_FIFO / WR_BP_FIFO

in: {rob_index,phy_adr,wr_data_prt}, push

out: o, full, partial_full, empty, pop

TURN_CTRL

in: rd_full, wr_full, rd_empty, wr_empty, batch_state

out: rd_wr_select, batch_dir

WLR_SEL

in: rd_stream, wr_stream, rd_wr_select

out: combined_stream

LOOKAHEAD

in: stream_head, bank_rdy[N_BANKS]

out: sel_pkt{rob_index,phy_adr,op}, shift

BANK_DEC

in: pkt(daddr)

out: bank_push_oh[N_BANKS], bank_req

MCC — per-bank cell (xN_BANKS)

BANK_REQ_REG

in: bank_req{rob_index,col,op,sram_addr}, wr_en, grant_oh

out: req_meta{op,rob_index,col,sram_addr}

BANK_STATUS_REG

in: fsm_wr{state,open_row,valid,pre_pending}, pre_propose

out: state[4], open_row, valid, pre_pending

ROW_COMPARATOR

in: pkt_row, open_row

out: is_hit, need_pre, need_act, need_cas

BANK_ELIG

in: need_*, can_act, can_cas, can_pre, row_valid, stall_acts

out: bank_go, bank_cmd(cas/act/pre), is_hit

BANK_FSM

in: cmd[3], grant_act/cas/pre, ap, can_cas, can_act, gate_rfc, gate_rfm, pre_pending

out: state[4], act_set, cas_set, pre_set, row_valid, cas_complete, bank_busy

MCC — arb tree

CMD_ARB

in: bank_go, bank_cmd, is_hit, cas_w, act_w, pre_w

out: win{go,cmd,is_hit}

BGB_ARB

in: bank_win[4]

out: bg_win{go,cmd,bank}

BG_ARB

in: bg_win[8], last_cas_bg

out: rank_cand{go,cmd,bg,bank}

RANK_ARB

in: cand, can_act_rank, can_cas_rank, p_bit

out: act_win, cas_win, pre_win[0:1] = {rdy,bg,bank}

MCC — schedule + phaser

SCHEDULER

in: act_win, cas_win, pre_win, maint flags, dqfree, turnaround, gear

out: cmd_0/1{vld,len,packet}, grant_act/cas/pre_oh, fsm_commit, cmd_issued

CMD_PHASER

in: cmd_0, cmd_1, gear

out: cmd_0_enc, cmd_1_enc, rdy0, rdy1

DFI_CMD_ENCODER

in: record, gear_cfg, ui_idx

out: dfi_pN{address[14],cs_n,cke,odt,parity}

OPCODE_ROM

in: cmd_type[5], rw

out: two_ui, ca_fix_ui0[14], ca_fix_ui1[14], field_sel, cs_pattern

FIELD_MUX

in: record{bg,bank,addr,ap,bl,cid}, ui_idx

out: ca_field[14]

CS_GEN

in: rank, valid, ui_idx

out: dfi_cs_n[N_RANKS]

MCC — write data path

WR_ACCUM

in: w_beats{data, strb, last}, narrow
out: line[512], dm[512/8], wd_wr_en, wd_slot

WL_LAUNCH

in: wd_slot, rob_index, tphy_wrlat, tphy_wrdata, cas_commit
out: dfi_wrdata_en, split_fire, wr_done{rob_index, pkt}

WR_SPLIT

in: line[512], gear
out: dfi_wrdata_pN[2*DQ], dfi_wrdata_mask_pN

MCC — read data path

RD_CAP

in: dbuf_addr, rob_index, tphy_rdlat, dfi_rddata_valid
out: cap_en, accum_load, rd_done{rob_index, pkt}

RD_ACCUM

in: dfi_rddata_pN[2*DQ], cap_en
out: line[512], rd_wr_en, dbuf_addr

RD_SPLIT

in: line[512]
out: r_beats{rdata, rlast}, r_valid

MCC — completion / tracking

COMPLETION

in: rd_done, wr_done
out: resp{rob_index, pkt_num, status}, resp_valid

WR_ROB

in: alloc{N}, wr_done
out: gen_idx, addr_offset, retire, done

MCC — scoreboard

GC

in: gear
out: gc[24]

TIMING_CONST

in: cfg_write
out: t_const[] (bc)

*_UPD

in: gc, t_const, cmd_issued
out: next_* [13] (per scope)

*_TREG + CMP

in: next_*, gc, we
out: can_act, can_cas, can_pre, can*_bgs, can*_rank

refi_cnt

in: gear, refs_b_done, rr_shift
out: ref_rdy, ref_force, o_ref_rdy

RAA_cnt

in: act_issue, rfm_done, ref_done, raaimt, raammt
out: rfm_rdy, rfm_force

MCC — maintenance

MAINT_ENG

in: cas_complete[N_BANKS], debt, raa, mr4_temp, dfi_init_complete
out: stall_acts, gate_rfc, gate_rfm, maint_cmd{type, ba}, maint_inject, init_done